

DTL.03.1 - Training mit J48: Aufgabenstellung

Mit Hilfe von Weka wurden für die Datensätze restaurant.csv und zoo.csv Entscheidungsbäume trainiert. Dazu kam der Klassifikator J48 (Implementierung des C4.5-Algorithmus) zum Einsatz. Als Testmodus wurde Use training set gewählt. Für jeden Datensatz werden der erzeugte Baum, die Fehlerrate und die Confusion Matrix angegeben sowie kurz interpretiert.

1. Restaurant-Datensatz

Modell

Algorithmus: J48 (C4.5), Parameter `-C 0.25 -M 2`

Trainingsdaten: 12 Instanzen, 11 Attribute

Klasse: Waiting (Yes/No)

Erzeugter Baum:

```
Patrons
|-- Some -> Yes
|-- Full -> No
\__ None -> No
```

Ergebnisse

- Korrekt klassifizierte Instanzen: 10 von 12 (83.3 %)
- Fehlerrate: 16.7 %
- Kappa: 0.667
- Confusion Matrix:

	Yes	No
Yes	4	2
No	0	6

Interpretation

Der Baum prüft zunächst das Attribut Patrons. Bei wenigen oder keinen Gästen (None) wird sofort No entschieden, bei Some immer Yes. Für volle Restaurants (Full) wird nach dem Pruning pauschal No ausgegeben – das führt zu zwei Fehlern. Der Baum ist dadurch sehr kompakt, aber etwas ungenauer als ein unpruned ID3-Baum, der alle 12 Trainingsinstanzen korrekt klassifiziert hätte.

2. Zoo-Datensatz

Modell

Algorithmus: J48 (C4.5), Parameter `-C 0.25 -M 2`

Trainingsdaten: 101 Instanzen, 18 Attribute

Klasse: type (Tierklasse)

Erzeugter Baum (gekürzt):

```
feathers <= 0
|   milk <= 0
|   |   backbone <= 0
|   |   |   airborne <= 0
|   |   |   |   predator <= 0
|   |   |   |   |   legs <= 2: shellfish
|   |   |   |   |   legs > 2: insect
|   |   |   |   predator > 0: shellfish
|   |   |   airborne > 0: insect
|   |   backbone > 0
|   |   |   fins <= 0
|   |   |   |   tail <= 0: amphibian
|   |   |   |   tail > 0: reptile
|   |   |   fins > 0: fish
|   milk > 0: mammal
feathers > 0: bird
```

Ergebnisse

- Korrekt klassifizierte Instanzen: 100 von 101 (99.0 %)
- Fehlerrate: 0.99 %
- Kappa: 0.987
- Confusion Matrix (vereinfacht):

	mammal	fish	bird	shellfish	insect	amphib.	reptile
mammal	41	0	0	0	0	0	0
fish	0	13	0	0	0	0	0
bird	0	0	20	0	0	0	0
shellfish	0	0	0	10	0	0	0
insect	0	0	0	0	8	0	0
amphib.	0	0	0	0	0	3	1
reptile	0	0	0	0	0	0	5

Interpretation

Der Baum trennt die Tierklassen nahezu perfekt. Die wichtigsten Merkmale sind **feathers**, **milk**, **fins**, **backbone** und **tail**. Nur ein Exemplar der Klasse **amphibian** wurde fälschlich als **reptile** klassifiziert, was biologisch nachvollziehbar ist (glaube ich). Damit liegt die Trainingsgenauigkeit bei 99 %.

Fazit

Beide Entscheidungsbäume zeigen, dass J48 (C4.5) mit wenigen, informativen Attributen sehr genaue Modelle erzeugt. Das Restaurant-Modell profitiert von mehr Detail (ohne Pruning), während der Zoo-Datensatz bereits mit starkem Pruning fast fehlerfrei klassifiziert wird. **(Im Github sind auch die beiden Logs, um alles nachzugucken)**

DTL.03.2: ARFF-Forma

1. Unterschied zwischen **nominal**, **ordinal/numeric** und **string**

- **Nominal:** Beschreibt Attribute mit diskreten, kategorialen Ausprägungen ohne natürliche Reihenfolge. Beispiel: {rot, blau, grün}, {Ja, Nein}. Nominale Werte können nur auf Gleichheit getestet werden ($=$, $\neq$), nicht auf größer/kleiner. Diese Art ist für Entscheidungsbäume wie ID3 ideal.
- **Ordinal / Numeric:** Numerische (oder geordnete) Attribute besitzen eine messbare Skala oder Rangfolge. Beispiele: Alter, Einkommen, Temperatur. Hier kann man arithmetische Operationen und Vergleiche ($<$, $>$) ausführen. ID3 kann mit rein numerischen Attributen nicht direkt umgehen – sie müssen zuerst in diskrete (nominale) Intervalle überführt werden.
- **String:** Freitext- oder Zeichenkettenattribute, z. B. Name oder Beschreibung. Diese enthalten meist viele einzigartige Werte und sind für Klassifikatoren ohne Bedeutung. In Weka sollten sie daher entfernt oder manuell in Kategorien überführt werden.

2. Konvertierung in das ARFF-Format

Die .csv-Dateien (restaurant.csv und zoo.csv) wurden in Weka geladen und in das ARFF-Format verwandelt.

Mein vorgehen in Weka:

1. Reiter **Preprocess** öffnen und Datei laden.

2. Unter **Filter** -> **Choose** den Filter `weka.filters.unsupervised.attribute.NumericToNominal` auswählen, um numerische Attribute in nominale umzuwandeln.
3. Nicht benötigte Attribute (z. B. `name` im Zoo-Datensatz) mit **Remove** löschen.
4. Unter **Class** die Zielvariable setzen (`Waiting` bei Restaurant, `type` bei Zoo).
5. Datei unter **Save as...** im Format `.arff` speichern.

3. Beispiel: `restaurant_id3.arff`

```
@RELATION restaurant_id3

@ATTRIBUTE Alternate    {Yes,No}
@ATTRIBUTE Bar          {No,Yes}
@ATTRIBUTE Fri/Sat     {No,Yes}
@ATTRIBUTE Hungry       {Yes,No}
@ATTRIBUTE Patrons      {Some,Full,None}
@ATTRIBUTE Price        {'$', '$$', '$$$'}
@ATTRIBUTE Raining      {No,Yes}
@ATTRIBUTE Reservation  {Yes,No}
@ATTRIBUTE Type          {French,Thai,Burger,Italian}
@ATTRIBUTE WaitEstimate {'0-10', '10-30', '30-60', '>60'}
@ATTRIBUTE Waiting      {Yes,No}

@DATA
Yes,No,No,Yes,Some,'$$$',No,Yes,French,'0-10',Yes
Yes,No,No,Yes,Full,'$',No,No,Thai,'30-60',No
No,Yes,No,No,Some,'$',No,No,Burger,'0-10',Yes
...
```

Dieser Datensatz enthält nur nominale Attribute und kann direkt mit dem ID3-Algorithmus trainiert werden.

4. Beispiel: `zoo_id3.arff`

Im Zoo-Datensatz wurde das Attribut `name` entfernt (String-Attribut) und das numerische Attribut `legs` in nominale Werte umgewandelt. Die Klasse ist `type`.

```
@RELATION zoo_id3

@ATTRIBUTE hair         {0,1}
@ATTRIBUTE feathers     {0,1}
@ATTRIBUTE eggs         {0,1}
@ATTRIBUTE milk         {0,1}
@ATTRIBUTE airborne     {0,1}
@ATTRIBUTE aquatic      {0,1}
```

```

@ATTRIBUTE predator      {0,1}
@ATTRIBUTE toothed       {0,1}
@ATTRIBUTE backbone      {0,1}
@ATTRIBUTE breathes      {0,1}
@ATTRIBUTE venomous      {0,1}
@ATTRIBUTE fins           {0,1}
@ATTRIBUTE legs           {0,2,4,5,6,8}
@ATTRIBUTE tail           {0,1}
@ATTRIBUTE domestic      {0,1}
@ATTRIBUTE catsize        {0,1}
@ATTRIBUTE type           {mammal,fish,bird,shellfish,insect,amphibian,reptile}

```

```

@DATA
1,0,0,1,0,0,1,1,1,1,0,0,4,0,0,1,mammal
0,0,1,0,0,1,1,1,1,0,0,1,0,1,0,0,fish
1,0,0,1,0,0,0,1,1,1,0,0,4,1,0,1,mammal
...

```

Ergebnis: Beide Datensätze (`restaurant_id3.arff` und `zoo_id3.arff`) können nun in Weka mit dem Entscheidungsbaum-Lerner **ID3** ohne Fehlermeldung trainiert werden.

Kurzinterpretation:

- `restaurant_id3.arff`: 11 Attribute, 12 Instanzen, alle nominal, Klasse `Waiting`.
 - `zoo_id3.arff`: 17 Attribute (nach Entfernen von `name`), 101 Instanzen, Klasse `type`.
 - Beide ARFF-Dateien sind vollständig ID3-kompatibel.
-

DTL.03.3 - Training mit ID3 und J48

Ziel

Für die in ARFF konvertierten Datensätze (`restaurant-id3.arff` und `zoo-id3.arff`) wurden Entscheidungsbäume mit den Lernverfahren ID3 und J48 (C4.5) trainiert und anschließend verglichen. Dabei wurden Baumstruktur, Fehlerrate und Confusion Matrix untersucht und mit den früheren Ergebnissen aus den CSV-Läufen verglichen.

Restaurant-Datensatz

ID3

```

Patrons = Some: Yes
Patrons = Full
|   Type = French: No
|   Type = Thai
|   |   Fri/Sat = No: No
|   |   Fri/Sat = Yes: Yes
|   Type = Burger
|   |   Alternate = Yes: Yes
|   |   Alternate = No: No
|   Type = Italian: No
Patrons = None: No

```

Ergebnisse:

- Korrekt klassifizierte Instanzen: 12 von 12 (**100 %**)
- Fehlerrate: 0 %
- Confusion Matrix:

$$\begin{bmatrix} 6 & 0 \\ 0 & 6 \end{bmatrix}$$

Interpretation: ID3 passt die Trainingsdaten perfekt an (keine Fehler), neigt aber zum *Overfitting*, da keine Pruning-Strategie angewendet wird.

J48

```

Patrons = Some: Yes
Patrons = Full: No
Patrons = None: No

```

Ergebnisse:

- Korrekt klassifizierte Instanzen: 10 von 12 (**83,3 %**)
- Fehlerrate: 16,7 %
- Confusion Matrix:

$$\begin{bmatrix} 4 & 2 \\ 0 & 6 \end{bmatrix}$$

Interpretation: J48 liefert einen kompakteren, geprunten Baum. Die Fehlerrate ist etwas höher, dafür ist die Struktur einfacher und besser generalisierbar. Die Ergebnisse stimmen mit den Läufen aus den CSV-Dateien überein.

Zoo-Datensatz

ID3

```
legs = 0
|  fins = 0
|  |  toothed = 0: shellfish
|  |  toothed = 1: reptile
|  fins = 1
|  |  eggs = 0: mammal
|  |  eggs = 1: fish
legs = 2
|  hair = 0: bird
|  hair = 1: mammal
legs = 4
|  hair = 0
|  |  aquatic = 0: reptile
|  |  aquatic = 1
|  |  |  toothed = 0: shellfish
|  |  |  toothed = 1: amphibian
|  hair = 1: mammal
legs = 5: shellfish
legs = 6
|  aquatic = 0: insect
|  aquatic = 1: shellfish
legs = 8: shellfish
```

Ergebnisse:

- Korrekt klassifizierte Instanzen: 101 von 101 (**100 %**)
- Fehlerrate: 0 %
- Confusion Matrix: perfekt diagonal (alle Klassen korrekt erkannt)

Interpretation: Der ID3-Baum ist sehr groß und trennt nahezu jedes Objekt einzeln. Dies führt zu einer perfekten Klassifikation der Trainingsdaten, zeigt aber starkes Overfitting.

J48

```
feathers = 0
|  milk = 0
|  |  backbone = 0
|  |  |  airborne = 0
|  |  |  |  predator = 0
|  |  |  |  |  legs = 0: shellfish
|  |  |  |  |  legs = 6: insect
```

```

|   |   |   |   predator = 1: shellfish
|   |   |   airborne = 1: insect
|   |   backbone = 1
|   |   |   fins = 0
|   |   |   |   tail = 0: amphibian
|   |   |   |   tail = 1: reptile
|   |   |   fins = 1: fish
|   milk = 1: mammal
feathers = 1: bird

```

Ergebnisse:

- Korrekt klassifizierte Instanzen: 100 von 101 (**99,0 %**)
- Fehlerrate: 1 %
- Confusion Matrix: nur 1 Verwechslung zwischen *amphibian* und *reptile*

Interpretation: J48 liefert einen stark reduzierten, übersichtlichen Baum mit minimalem Fehler. Das Pruning führt zu besserer Generalisierung bei fast gleicher Genauigkeit.

Gesamtvergleich

- **ID3:** perfekte Ergebnisse auf Trainingsdaten (0 % Fehler), aber Gefahr des Overfittings.
- **J48:** kompaktere Bäume mit leicht höherer Fehlerrate, aber besserer Generalisierbarkeit.
- Beide Verfahren wählen sinnvolle, stark trennende Attribute als Wurzel:
 - Restaurant: Patrons
 - Zoo: Feathers bzw. Legs

Fazit: ID3 erzeugt sehr tiefe, perfekte Bäume für Trainingsdaten, während J48 durch Pruning kompaktere Modelle liefert. Die Ergebnisse der ARFF-Datensätze stimmen 'weitgehend' mit den Läufen auf den ursprünglichen CSV-Dateien überein.